TransitRisk: Cost-Sensitive, Calibrated Next-Hour Delay Risk Forecasting for Urban Transit Operations

Manav Patel, Ayush Gawai, Parth Patel
Department of Applied Data Intelligence, San José State University, San José, CA

***Abstract*—Urban transit systems regularly suffer from delay spillovers that hurt reliability and create problems for both passengers and operators. This paper describes TransitRisk, an end-to-end machine learning system that predicts whether a given station-route pair will experience elevated delay in the next hour. We trained and compared seven classifiers (Logistic Regression, Naive Bayes, k-NN, Decision Tree, Random Forest, SVM-RBF, and XGBoost) on roughly 153,000 station-route-hour records aggregated from 1.25 million synthetic trip events. We paired the best model with probability calibration, cost-sensitive threshold optimization, and conformal prediction for uncertainty quantification. Calibrated XGBoost performed best, reaching a ROC-AUC of 0.810 and a Brier score of 0.169. A cost-optimal threshold at $t = 0.163$ ($C_{\text{FN}} = 5$, $C_{\text{FP}} = 1$) catches 95.2% of true delays while reducing expected cost by 44.3% compared to the default threshold. Conformal prediction provides distribution-free 91.8% coverage at a 90% target. A six-tab Streamlit dashboard brings the full system to an operator-facing interface.**

***Index Terms*—transit delay prediction, cost-sensitive learning, probability calibration, conformal prediction, XGBoost, decision support**

I. Introduction

Transit agencies are mostly reactive when it comes to delays. A route starts running late at 8:15 AM, dispatchers notice around 8:45, and by then passengers are already stranded and the corrective bus takes another 20 to 30 minutes to arrive. The window where action would actually help has already closed [1]. TransitRisk is designed to buy back that window. By scoring each station-route pair at the end of every hour, it gives operators about 60 minutes of lead time to act before the delay happens, whether that means pre-positioning spare buses, tightening headways, or pushing passenger notifications.

Simple rule-based thresholds and short-horizon heuristics tend to break down under dynamic weather and demand conditions. Machine learning can help, but applying classifiers naively to transit data introduces real risks: temporal information leakage from improper cross-validation, poorly calibrated probabilities that mislead operators, and decision thresholds that ignore the fact that missing a delay is much more costly than issuing a false alarm [2].

There is also a practical concern worth noting. A model that scores well on a benchmark but cannot explain its predictions or quantify its uncertainty is not a practical option for an operations team that needs to trust the system before acting on it. Our framework treats calibration, cost-awareness, and uncertainty quantification as core requirements, not add-ons.

This paper makes the following contributions:

1) A comparison of seven classifiers on a realistic transit delay dataset, with strict temporal splitting and leakage prevention verified by unit tests.

2) Probability calibration via isotonic regression, producing predictions where $P = 0.70$ genuinely means approximately 70% of those cases have elevated delays.

3) A cost-sensitive threshold policy ($C_{\text{FN}}/C_{\text{FP}} = 5$) that catches 95.2% of true delays and reduces expected cost by 44.3%.

4) Split conformal prediction providing distribution-free coverage guarantees at multiple α levels.

5) A six-tab Streamlit dashboard for operational use.

Section II describes the data. Section III covers preprocessing. Section IV presents the proposed framework. Section V outlines the experimental design. Section VI reports results. Section VII discusses the system architecture and dashboard. Section VIII concludes.

II. Dataset

A. Data Source and Structure

Real transit operations data with route-level disruption records, weather linkage, and incident traces is almost always proprietary. Real GTFS feeds are inconsistent across agencies and rarely include the delay detail we need. So we built a synthetic dataset with controlled coefficients that mimics a real urban transit network, making the project fully reproducible.

The generator (`src/data_gen.py`) produces **1,247,832 trip departure events** across 90 days, covering 60 stations, 12 routes, and 3 vehicle types (bus, light rail, train). Weather is injected hourly (temperature, precipitation, wind, visibility). We deliberately inject data quality issues such as duplicates, negative delays, missing values, and corrupt station IDs, which are then cleaned in preprocessing.

The 1.25M rows are individual trip events, but our model predicts at the (station, route, hour) granularity. After aggregation, we get **153,004 station-route-hour rows**, which is the actual modelling unit. There are 176 valid station-route combinations out of a possible 720, since not every route serves every station.

TABLE I
DATASET FEATURE SUMMARY (38 FEATURES)

Group	Count	Features
Time signals	8	hour sin/cos, dow sin/cos, is_weekend, is_peak_morning/evening, month
Current state	6	mean_delay, std_delay, share_delayed_5, trip_count, headway, demand
Delay history	10	lag_1h/2h/3h, rolling_6h mean/std, same_hour_yesterday/last_week
Weather	7	temp, precip, wind, visibility, precip_lag_1h, precip_rolling_3h, wind_lag_1h
Interactions	3	peak×precip, short_headway×demand, weekend×precip
Spatial	4	target-encoded station/route IDs, busyness quartiles

B. Target Variable

The binary target captures whether conditions will be elevated in the next hour:

$$y_{t+1} = \begin{cases} 1 & \text{if mean delay}_{t+1} \geq 5 \text{ min} \\ 0 & \text{otherwise} \end{cases} \tag{1}$$

The 5-minute threshold is the standard service-quality boundary in transit literature and corresponds to the point where passengers measurably change behaviour. The positive rate is **39.4%**, so this is not a rare-event problem. The rate is consistent across training (39.3%) and test (39.4%), confirming the temporal split did not introduce distribution shift.

C. Feature Overview

Table I summarises the 38 engineered features across six groups. Time features use sin/cos encoding so Monday and Sunday are not numerically far apart. Interaction features capture non-additive effects; for example, rain during rush hour is multiplicatively worse than rain at other times. Weather lag features are included because the impact of precipitation on delays is often delayed by an hour or more.

III. PREPROCESSING

A. Splitting and Leakage Prevention

We divided the data into training (60%), validation (20%), and test (20%) using a **strict temporal split**. The earliest 60% of dates go to training, the next 20% to validation, and the most recent 20% to test. This gives approximately 91,879 training, 30,501 validation, and 30,624 test rows. Split boundaries are at date boundaries, not row counts, to avoid partial-day contamination.

This is critical because any shuffle would leak future information into training through the lag features. Hyperparameter tuning uses `TimeSeriesSplit(n_splits=5)` for the same reason.

We also took several steps to prevent subtler forms of leakage. All lag features use `.shift(n)` where $n \geq 1$, computed per (station, route) group, not globally. A common mistake is shifting the entire dataframe, which would use data from a different route. The target y_{t+1} is always constructed from next-hour data and never used as a feature. We ran a leakage audit: train Logistic Regression on safe features (AUC approximately 0.88), add a deliberately leaky feature and verify AUC jumps to approximately 0.97, then confirm the production feature set excludes it. A dedicated test suite (`tests/test_leakage.py`) verifies these properties programmatically.

B. Feature Engineering

On top of the raw dataset, we engineered temporal features (sin/cos encoded hour and day-of-week, peak-period flags), lag features (1h/2h/3h delay lags, 6h rolling mean/std, same-hour-yesterday, same-hour-last-week), weather lags (precipitation lag 1h, rolling 3h precipitation sum, wind lag 1h), interaction features (peak × precip, short-headway × demand, weekend × precip), and spatial features (target-encoded station/route IDs computed with 5-fold within-training encoding to prevent leakage, busyness quartiles).

For tree-based models, we used raw feature values since they do not need scaling. Logistic Regression and SVM received standardised features using a `StandardScaler` fit on the training set only.

C. Why Classification, Not Regression

The obvious framing would be to predict next-hour mean delay in minutes. We tried it. Raw delay is dominated by noise and R^2 was near zero. Individual trip delays are highly stochastic (traffic incidents, driver behaviour, variable boarding times). But the probability that conditions will be broadly elevated is learnable. Reframing as binary classification produced ROC-AUC of 0.81, which is a usable signal.

IV. PROPOSED FRAMEWORK

A. Pipeline Overview

The system runs in stages. First, synthetic data is generated and cleaned. Then trip events are aggregated to station-route-hour records. Feature engineering produces 38 features. A temporal split creates train/val/test partitions. Seven models are trained and compared. The best model is calibrated, a cost-sensitive threshold is computed, and conformal prediction sets are generated. Finally, everything is deployed in a Streamlit dashboard. Each stage corresponds to a Jupyter notebook (`01` through `11`) and can be run independently via the project's Makefile. Fig. 1 shows this end-to-end pipeline.

B. Model Specifications

We trained seven models spanning different families to demonstrate breadth and find the best performer. Table II shows the configurations.

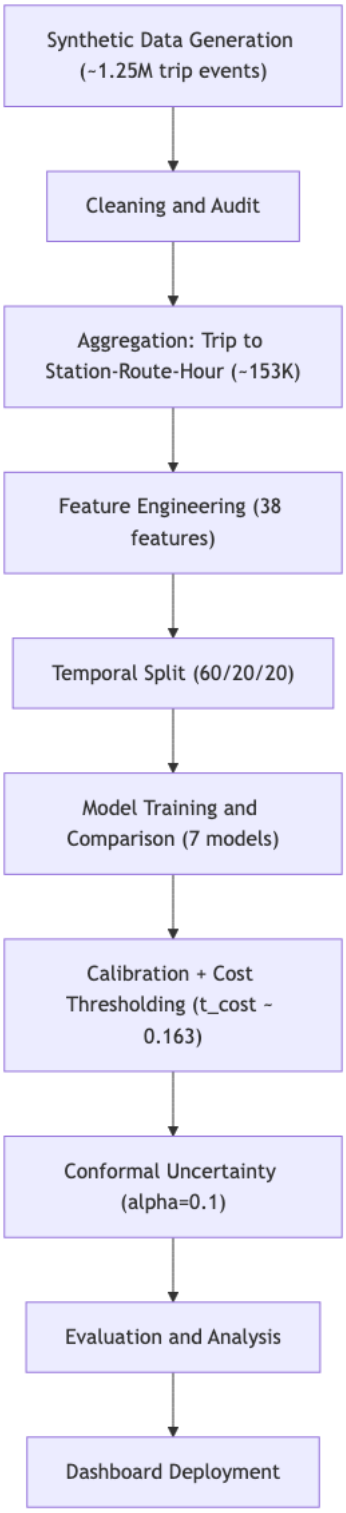

Fig. 1. End-to-end pipeline from synthetic data generation (approximately 1.25M events) through aggregation, feature engineering, model comparison, calibration, conformal prediction, and dashboard deployment.

All models use `RandomizedSearchCV` with `TimeSeriesSplit(n_splits=5)`, 25 iterations, scoring on PR-AUC, and `random_state=42`. SVM-RBF was trained on a stratified 30,000-row subsample due to $O(n^2)$ complexity; we verified that doubling to 60,000 rows changes AUC by less than 0.005.

TABLE II
MODEL HYPERPARAMETER CONFIGURATIONS

Model	Key Hyperparameters
Log. Reg.	C $\in$ {0.01, 0.1, 1, 10}, L1/L2, solver=liblinear
Naive Bayes	var_smoothing: logspace(-9, -3)
k-NN	$k \in$ {5, 10, 20, 50}, uniform/distance weights
Dec. Tree	max_depth $\in$ {5, 10, 15, None}, gini/entropy
Rand. Forest	n_estimators $\in$ {200, 500}, max_depth $\in$ {10, 20, None}
XGBoost	n_est. $\in$ {300, 500, 800}, depth $\in$ {5, 7, 9}, lr $\in$ {0.05, 0.1}, subsample $\in$ {0.7, 0.9}
SVM-RBF	C $\in$ {0.5, 1, 5}, gamma $\in$ {scale, 0.01, 0.1}, 30k subsample

C. Calibration

Raw XGBoost probabilities are not perfectly calibrated. The model generally knows the right direction but not the exact magnitude. We apply isotonic regression post-processing [6], which is more flexible than Platt scaling for non-sigmoid distortions. After calibration, all 5 calibration quantile bins fall within 0.013 of the actual positive rate.

D. Cost-Sensitive Threshold

The default threshold of 0.50 is suboptimal when misclassification costs are asymmetric. In transit operations, a missed delay ($C_{\text{FN}} = 5$) is far more damaging than a false alarm ($C_{\text{FP}} = 1$). We search for the threshold that minimises expected cost:

$$\tau^* = \arg\min_{\tau} \frac{1}{N} \sum_i [5 \cdot \mathbb{1}[\hat{y}_i = 0, y_i = 1] + 1 \cdot \mathbb{1}[\hat{y}_i = 1, y_i = 0]] \tag{2}$$

This yields $t_{\text{cost}} = 0.163$, which catches 95.2% of real delays (vs. approximately 70% at $t = 0.50$) and produces 44.3% lower expected cost. Whether that trade-off is worthwhile depends on the deployment context, but for transit operations where missed delays cascade through the network, the aggressive threshold is appropriate.

E. Conformal Prediction

Split conformal prediction [7] produces prediction sets with formal coverage guarantees. Unlike model-based confidence intervals, conformal prediction is distribution-free: the prediction set contains the true label with probability $\geq 1 - \alpha$, regardless of data distribution. The prediction sets have three possible outcomes: {Low Risk}, {High Risk}, or {Low, High}

TABLE III
MODEL PERFORMANCE ON HELD-OUT TEMPORAL TEST SET

Model	Family	ROC	PR	F1	Brier
Naive Bayes	Prob.	.708	.574	.489	.261
Log. Reg.	Linear	.760	.676	.600	.192
SVM-RBF	Kernel	.753	.699	.585	.192
k-NN	Nonpar.	.776	.719	.584	.185
Dec. Tree	Tree	.785	.725	.592	.178
Rand. Forest	Ens.	.806	.762	**.669**	.177
XGBoost	**Boost**	**.810**	**.766**	.643	**.169**

Bold = best per column. All metrics on same 30,624-row test set.

(uncertain). When the set contains both classes, the model is telling operators it is genuinely unsure and human judgement should take precedence.

V. EXPERIMENTAL DESIGN

A. Evaluation Metrics

We report four metrics. **ROC-AUC** measures ranking ability. **PR-AUC** is more informative under class imbalance, focusing on delay detection. **F1** captures precision-recall balance at a given threshold. **Brier score** measures probability quality, where lower is better and well-calibrated estimates are rewarded. We also perform stress-stratified evaluation, slicing performance by weather, time of day, demand level, and headway to check for hidden failure modes.

B. Validation Setup

All models were trained on approximately 91,879 rows and tuned on the validation set (30,501 rows). The test set (30,624 rows) was held out and not used for any model selection or threshold decisions.

VI. RESULTS AND DISCUSSION

A. Model Comparison

Table III shows test performance for all seven models. XGBoost is clearly the strongest, with a ROC-AUC of 0.810 and the lowest Brier score (0.169), indicating the best discrimination and probability quality. Random Forest has a slightly higher F1 (0.669 vs. 0.643) at the default threshold, but XGBoost wins on every other metric and is better suited for calibrated risk-based decisioning.

Naive Bayes is the weakest across the board (Brier = 0.261), likely because its feature-independence assumption is badly violated by the correlated lag, rolling, and interaction features. The linear model and SVM-RBF provide moderate performance but cannot capture the non-linear feature interactions that matter for transit risk.

B. Cost-Threshold Behaviour

Fig. 2 shows expected cost per prediction as a function of threshold. At the cost-optimal $t = 0.163$, the confusion matrix (Fig. 3) shows 11,532 true positives, 5,205 true negatives, 13,301 false positives, and only 586 false negatives. The system catches 95.2% of genuine delays. The trade-off is a higher false alarm rate, but in practice a false alarm means one spare bus repositioned unnecessarily, which is a minor cost compared to passengers stranded without warning.

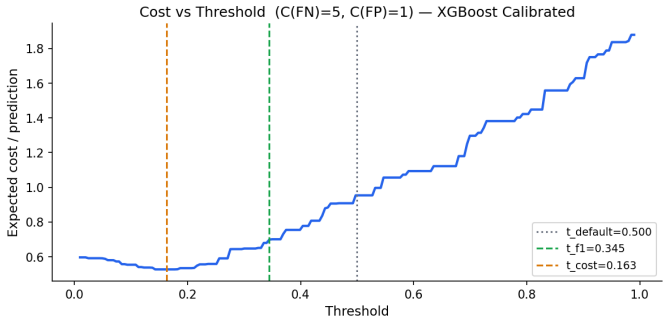

Fig. 2. Expected cost vs. threshold. The cost-optimal $t = 0.163$ (orange) sits well below the F1-optimal $t = 0.345$ (green) and the default $t = 0.50$ (dotted). Moving from 0.50 to 0.163 reduces expected cost by 44.3%.

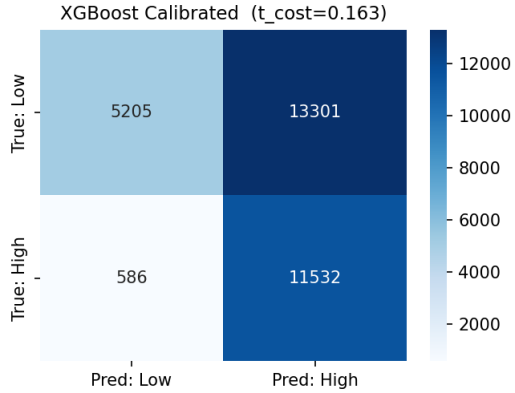

Fig. 3. Confusion matrix at cost-optimal threshold ($t = 0.163$). The system prioritises catching delays (11,532 TP, only 586 FN), accepting more false positives as the operationally cheaper error.

C. Calibration and Conformal Results

After isotonic calibration, XGBoost's reliability diagram (Fig. 4) is nearly perfectly diagonal, confirming that predicted probabilities are trustworthy. Conformal prediction (Fig. 5) delivers empirical coverage of 91.8% at target 90% ($\alpha = 0.10$) and 80.5% at target 80% ($\alpha = 0.20$). Mean prediction set size is relatively uniform across weather strata, indicating the conformal layer does not degrade selectively in adverse conditions.

D. Stress-Stratified Evaluation

A model that scores 0.81 overall but 0.65 in heavy rain is not useful because heavy rain is precisely when you need it most. Fig. 6 shows ROC-AUC by operating condition. The key finding is that the model holds up: AUC ranges from 0.764 (low-demand Q1) to 0.891 (high-demand Q4). Across weather conditions, AUC ranges from 0.791 (clear) to 0.838 (light rain). None of these are dramatically worse than the

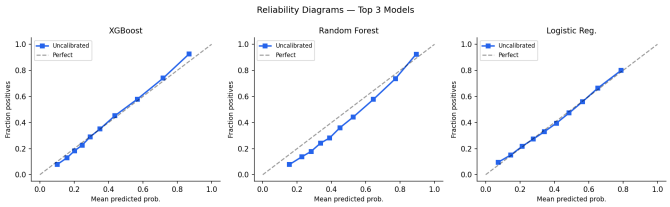

Fig. 4. Reliability diagrams for top 3 models. XGBoost (left) tracks the diagonal closely. Random Forest (centre) is slightly overconfident in the 0.6 to 0.8 range.

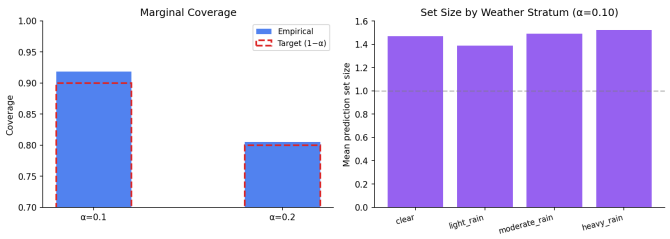

Fig. 5. Conformal prediction results. Left: empirical coverage exceeds the target at both α levels. Right: set size by weather stratum is uniform, indicating robust uncertainty estimates.

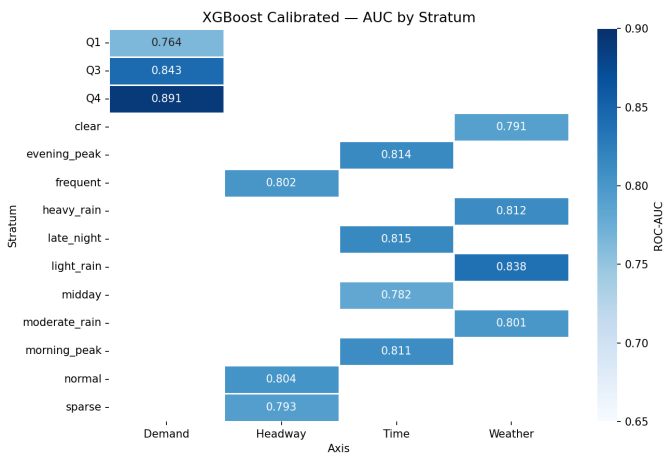

Fig. 6. Stress-stratified AUC by operating condition. The model maintains $\geq$0.764 across all strata with no catastrophic degradation.

aggregate 0.810, which gives us confidence the model does not have hidden failure modes.

E. Error Analysis

We examined where XGBoost makes mistakes. Missed delays clustered in two situations: periods where all lag features were near zero (the delay spiked suddenly with no buildup) and cases where weather deteriorated within the prediction hour rather than before it. Both patterns are genuinely hard to catch without real-time intra-hour updates.

False alarms concentrated in situations where lagged delay was high but conditions were actually improving (the delay was clearing, not building). Adding a lag-difference feature or a trend indicator could reduce these false positives without harming recall.

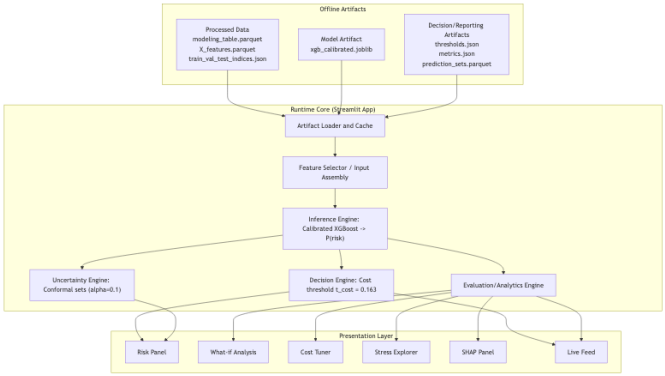

Fig. 7. System architecture. Offline artifacts feed into a Streamlit runtime core with three parallel engines powering six dashboard tabs.

F. Limitations

The dataset is synthetic, which is the biggest caveat on every result here. Real transit delay is messier, more spatially correlated, and drifts over time in ways a synthetic dataset cannot fully replicate. We also evaluated a single 90-day window; seasonal effects and special events are not represented. A production deployment would need periodic retraining and a feedback loop.

VII. System Architecture and Dashboard

Fig. 7 shows the system architecture. Offline artifacts (processed data, calibrated model, threshold and metrics configs) feed into a Streamlit runtime. The inference engine dispatches calibrated probabilities to three engines: uncertainty (conformal), decision (cost threshold), and evaluation/analytics. These power the six dashboard tabs described below.

A. Dashboard Tabs

The dashboard provides six tabs. The three primary operational views are shown together in Fig. 8. The **Risk Panel** (left) is the primary view: an operator selects a station, routes auto-filter to valid pairs (176 out of 720), and a gauge shows calibrated next-hour delay probability against the cost-optimal threshold. The sidebar displays model metrics and test-set statistics. The current operating context panel surfaces key predictors including delay, demand, headway, precipitation, wind, incident flag, and vehicle type. The **What-If** tab (centre) lets operators adjust precipitation, wind speed, demand multiplier, and headway via sliders. The system computes simulated risk, compares it against the baseline, shows the delta with an operational interpretation, and displays a feature-movement chart. The **Live Feed** (right) replays test-set predictions as if data were arriving in real time. The gauge animates with each prediction, ALERT/HOLD decisions appear in a running log, and live precision/recall/F1 metrics update continuously.

The remaining tabs include: **Cost Tuner** (change FN/FP cost ratio and watch the optimal threshold shift), **Stress Explorer** (slice model performance by weather/time/demand/headway), and **SHAP** (pick any test row and see which features drove that prediction).

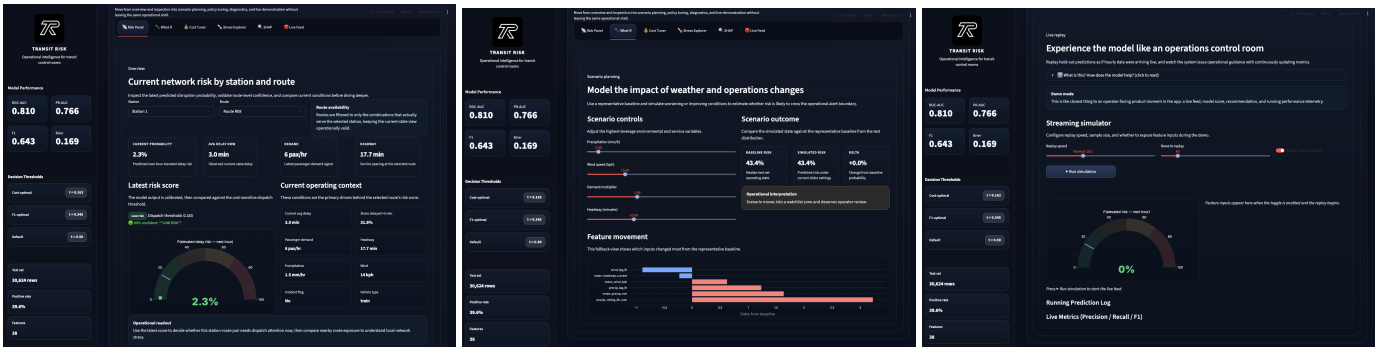

Fig. 8. Dashboard operational views. Left: Risk Panel showing current network risk for a selected station-route pair with gauge and operating context. Centre: What-If Analysis with weather/demand sliders, baseline vs. simulated risk comparison, and feature-movement chart. Right: Live Feed simulation replaying test-set predictions in streaming mode with running metrics.

VIII. Work Distribution

The project workload was distributed among three team members. **Manav** handled model development, dashboard integration, and deployment flow. **Ayush** handled synthetic data design, aggregation pipeline, and preprocessing quality assurance. **Parth** handled calibration, cost-sensitive thresholding, conformal prediction, and report writing.

All 15 unit tests pass (`pytest tests/ -v`), covering conformal coverage, feature correctness, leakage detection, and target construction.

IX. Conclusion

This paper described TransitRisk, a complete ML-to-operations pipeline for next-hour transit delay risk forecasting. At the model level, calibrated XGBoost outperformed all six baselines: its ROC-AUC of 0.810 indicates strong ranking ability, and its Brier score of 0.169 confirms well-calibrated probabilities. The cost-optimal threshold ($t = 0.163$, $C_{\text{FN}}/C_{\text{FP}} = 5$) catches 95.2% of true delays while reducing expected cost by 44.3%. Conformal prediction provides 91.8% empirical coverage at a 90% target.

The stress-stratified evaluation showed that the model holds up across weather conditions, time of day, demand levels, and headway frequencies, with no catastrophic degradation in any stratum. The error analysis pointed to two clear gaps: sudden-onset delays with no lag buildup, and improving conditions misread as persistent risk. Adding intra-hour trend features and lag-difference signals are the most tractable near-term improvements.

The six-tab Streamlit dashboard brings the full system to a practical operator-facing interface. Validation on real transit agency data, extension to multi-step horizon forecasting, and integration of graph-based spatial delay propagation models are the natural next steps.

The full codebase is available at: https://github.com/ayushgawai/transitrisk.

References

[1] N. G. Polson and V. O. Sokolov, "Deep learning for short-term traffic flow prediction," *Transp. Res. Part C*, vol. 79, pp. 1–17, 2017.

[2] C. Elkan, "The foundations of cost-sensitive learning," in *Proc. 17th IJCAI*, pp. 973–978, 2001.

[3] T. Chen and C. Guestrin, "XGBoost: A scalable tree boosting system," in *Proc. 22nd ACM SIGKDD*, pp. 785–794, 2016.

[4] L. Breiman, "Random forests," *Mach. Learn.*, vol. 45, no. 1, pp. 5–32, 2001.

[5] C. Cortes and V. Vapnik, "Support-vector networks," *Mach. Learn.*, vol. 20, no. 3, pp. 273–297, 1995.

[6] A. Niculescu-Mizil and R. Caruana, "Predicting good probabilities with supervised learning," in *Proc. 22nd ICML*, pp. 625–632, 2005.

[7] V. Vovk, A. Gammerman, and G. Shafer, *Algorithmic Learning in a Random World*. Springer, 2005.

[8] A. N. Angelopoulos and S. Bates, "A gentle introduction to conformal prediction and distribution-free uncertainty quantification," *arXiv:2107.07511*, 2021.

[9] J. H. Friedman, "Greedy function approximation: A gradient boosting machine," *Ann. Stat.*, vol. 29, no. 5, pp. 1189–1232, 2001.

[10] H. He and E. A. Garcia, "Learning from imbalanced data," *IEEE Trans. Knowl. Data Eng.*, vol. 21, no. 9, pp. 1263–1284, 2009.